

CRC 2026 - Technical report

DRAFT - Participant identity and personal review are pending.

Name: PENDING

School: PENDING

Faculty: PENDING

Goal: drive using onboard camera, LiDAR and wheel odometry. This submission prioritizes a small, explainable controller and conservative stopping. It does not claim complete track coverage or a competition score.

Evidence policy: results in this document come only from recorded driver telemetry and images.

Synthetic unit tests establish isolated behaviour, not successful driving. Missing evaluation results are stated explicitly.

Submission status: the participant must review this report, complete the identity and AI disclosure, and record the required continuous English video.

System architecture

The Python package `crc_solution` is independent of the official simulator package. One node owns motion commands. No simulator entity service, ground-truth stream or route-coordinate file is used.

Image and calibration -> lane and sign observations

LiDAR -> swept body corridor clearance

Odometry -> heading, stationary speed and travelled distance

Observations -> safety and traffic behaviour -> curvature control -> `/cmd_vel`

A wall-clock timer executes the control loop so loss of the simulation clock still produces zero velocity. Behaviour durations use simulation time. Images are processed only when a new frame arrives; repeated control ticks do not count as fresh detections.

The launch interface is `ros2 launch crc_solution run.launch.py`. The simulator is already running, and dependencies and workspace setup are completed before this command.

Lane perception and steering

White lane markings are selected using brightness, low saturation and local contrast. Multiple horizontal image bands provide candidates. Pairs are checked against projected lane width; a single visible boundary gives lower confidence.

The stock camera intrinsics and mounting height project pixels onto a locally flat ground plane. A line fit estimates lateral offset and heading. Consistency across bands determines confidence. The inferred target is in the robot frame, not a world coordinate.

The angular command is linear speed multiplied by a curvature estimate toward the look-ahead target. Positive target offset is left of the robot, giving positive angular velocity. Speed falls with curvature and lower confidence.

Brief missing markings reuse the most recent lane estimate with an odometry yaw correction. This prediction is bounded in time. A longer perception loss stops motion. Ground projection is approximate on a ramp; this is a known failure mode.

Traffic and obstacle behaviour

STOP candidates use red colour and a compact polygonal shape. Warning triangles are rejected. Several distinct frames are required. Approximate apparent sign size triggers the stop approach. The hold timer starts when measured wheel-odometry speed is almost zero and runs for at least two simulation seconds.

Traffic lights require a coloured compact region and a dark vertical housing. Temporal confirmation reduces single-frame errors. Red and yellow cause stopping before a junction. A short odometry-based crossing interval permits clearing a junction already entered on green.

LiDAR rays are transformed using the scan angle origin and increment. Returns in the projected body corridor cause stopping. Invalid front-sector data is not interpreted as free space. Clearance must persist before motion resumes.

Safety stopping has priority over forward progress. The driver does not recognize every sign type and does not overtake. These limitations must remain visible in the participant explanation.

Environment and reproducibility

The supplied environment uses ROS 2 Humble and Gazebo Classic 11 in Docker. The host in this session is Windows with an existing Ubuntu 20.04 VMware guest, configured for 8 GB RAM and 8 virtual CPUs.

The official simulator world, model and configuration files are protected by SHA-256 hashes. `tools/check_rules.py` verifies them and scans the submitted controller for forbidden interfaces. The Dockerfile uses HTTPS repositories to accommodate this network.

Build and install instructions are in README.md. Runtime dependencies are ROS Python bindings, `cv_bridge`, NumPy and OpenCV. Analysis uses matplotlib and PyMuPDF on the host.

Clean-environment validation must be distinguished from a fresh build in a reused container. A clean-machine claim is only justified after its corresponding check has actually run. See PROGRESS.md for the current status.

Validation method

Unit tests cover projected lane geometry, dim lanes, blank imagery, STOP versus warning triangles, lamp housing, missing sensors, invalid LiDAR, scan angle conventions, bounded lane prediction, obstacle clearance dwell and traffic behaviour.

Simulation evaluation targets three independent runs at `track_scale=1.0` with all props, signs and lights enabled, plus two valid changed start poses. Each evaluation is bounded to five simulation minutes. Different runs may have different traffic phases.

CSV telemetry contains time, state, commanded and measured speed, odometry, lane confidence, estimated lane offset and detected traffic controls. Annotated frames are saved periodically and at state transitions.

`analysis/summarize.py` produces all metric plots from the original CSVs. Estimated lane RMS is not official ground-truth lane RMS; integrated wheel distance is not the route-completion score. No collision or traffic compliance count is inferred without a verified annotation method.

Recorded results - pending

No real-run telemetry was available when this PDF was generated. No distance, lane RMS, success rate, collision count or competition score is claimed.

Run the evaluation, export `/tmp/crc_results` from the container, execute `analysis/summarize.py` results, then regenerate this PDF with `analysis/build_report.py`. Do not replace missing observations with expected values.

Evidence figures - pending

The plot page is intentionally empty until measured telemetry exists. The analysis script creates an odometry trajectory, linear-speed plot, camera lane-offset plot and perception/steering plot.

Images in the simulator documentation are reference material, not results of this solution. Only frames captured during this solution running may be described as experimental evidence.

Limitations and next experiments

Tight lane clearance amplifies calibration and boundary-selection errors. Intersections, crosswalk stripes and multiple parallel markings can create ambiguous candidates. The baseline selects visible continuity; it has no full route planner.

Camera projection assumes approximately level ground. Ramp pitch and tunnel flicker can reduce confidence. A stop is safer than a blind recovery but reduces distance covered.

Small or oblique STOP plates and lamps can be missed or misclassified. Temporal confirmation cannot recover an object that is never detected. Red-light stopping can persist if the lamp is lost from view.

Stationary obstacles stop progress because overtaking is disabled. Bus-stop, highway and warning-sign-specific actions are incomplete. There is no validated automatic FINISH detector.

Next experiments should focus on real failure frames, camera-ground calibration and short controlled approaches to each traffic object before attempting a more complex lane-change controller.

AI assistance and personal review

AI-generated work in this session: the initial package implementation, unit tests, analysis utilities, environment automation, README, report generator and video rehearsal outline. The AI also inspected supplied simulation assets during development.

Participant-authored or modified work: PENDING participant declaration. Record exactly what you changed, why you changed it, and what you personally tested. Do not claim sole authorship of generated code.

Participant understanding checklist: explain the sign of angular velocity; show how lane pixels become a target; explain sensor timeout and STOP hold; change max_speed live; distinguish an estimated metric from official scoring; describe at least one observed failure.

References: official UEH CRC 2026 Simulation Guideline and Simulation Pack supplied by the organizer; ROS 2 Humble interfaces used by the starter; Docker Ubuntu installation documentation, <https://docs.docker.com/engine/install/ubuntu/>.

This draft must be personally reviewed before submission. The participant must supply the unedited six-minute video and confirm the repository identity and submission deadline.